



Ejercicios 4. C# Métodos y Clases.

Ejercicio 1. Crea una clase llamada Cuenta que tendrá los siguientes atributos: titular y cantidad (puede tener decimales). El titular será obligatorio y la cantidad es opcional. Crea dos constructores que cumpla lo anterior.

Crea sus métodos get, set y toString. Tendrá dos métodos especiales:

- Ingresar(double cantidad): se ingresa una cantidad a la cuenta, si la cantidad introducida es negativa, no se hará nada.
- Retirar(double cantidad): se retira una cantidad a la cuenta, si restando la cantidad actual a la que nos pasan es negativa, la cantidad de la cuenta pasa a ser 0.

Ejercicio 2. Nos piden realizar una agenda telefónica de contactos. Un contacto está definido por un nombre y un teléfono (No es necesario validar). Un contacto es igual a otro cuando sus nombres son iguales. Una agenda de contactos está formada por un conjunto de contactos (Piensa en que tipo puede ser). Se podrá crear de dos formas, indicándoles nosotros el tamaño o con un tamaño por defecto (10). Los métodos de la agenda serán los siguientes:

- añadirContacto(contacto c): añade un contacto a la agenda, sino se puede meter más a la agenda se indicará por pantalla. No se pueden meter contactos que existan, es decir, no podemos duplicar nombres, aunque tengan distinto teléfono.
- existeContacto(contacto c): indica si el contacto pasado existe o no.
- listarContactos(): Lista toda la agenda
- buscarContactos (string nombre): busca un contacto por su nombre y muestra su teléfono.
- eliminarContacto(contacto c): elimina el contacto de la agenda, indica si se ha eliminado o no por pantalla.
- agendaLlena(): indica si la agenda está llena.
- huecosLibres(): indica cuantos contactos más podemos meter.

Crea un menú con opciones por consola para probar todas estas funcionalidades.

Ejercicio 3. Desarrolla una clase Cancion con los siguientes atributos:

- titulo: una variable string que guarda el título de la canción
- autor: una variable string que guarda el autor de la canción.

Crea los siguientes métodos:

- Canción (string, string): constructor que recibe como parámetros el título y el autor de la canción (por este orden).
- Cancion (): constructor predeterminado que inicializa el título y el autor a cadenas vacías .
- dameTitulo(): devuelve el título de la canción.
- dameAutor (): devuelve el autor de la canción.
- ponTitulo (string): establece el título de la canción
- ponAutor (string): establece el autor de la canción.

Ejercicio 4. Desarrolla una clase CD con los siguientes atributos:

- canciones: un array de objetos de la clase Cancion.
- contador: la siguiente posición libre del array canciones. y los siguientes métodos:
- CD(): constructor predeterminado (creará el array canciones).
- numeroCanciones(): devuelve el valor del contador de canciones.
- dameCancion(int): devuelve la Cancion que se encuentra en la posición indicada.
- grabaCancion(int, Cancion): cambia la Cancion de la posición indicada por la nueva Cancion proporcionada.
- agrega(Cancion): agrega al final del array la Cancion proporcionada.
- elimina(int): elimina la Cancion que se encuentra en la posición indicada.

Ejercicio 5. Implementar una clase Juego con las siguientes características:

Atributos

- Tiene como atributo público un entero que indica el número de vidas que le quedan al jugador en la partida actual.

Métodos

- Tiene como método el constructor que acepta un parámetro de tipo entero que indica el número de vidas iniciales con las que parte el jugador.
- Tiene un método MuestraVidasRestantes que visualiza por pantalla el número de vidas que le quedan al jugador en la partida actual.
- Además, esta clase tiene también el método main que debe realizar lo siguiente:
 - o Crea una instancia de la clase Juego indicando que el número de vidas es 5.
 - o Llama al método MuestraVidasRestantes del objeto creado.
 - o Resta una vida al valor del atributo con las vidas y vuelve a llamar a MuestraVidasRestantes.
 - o Crea otra instancia de la clase Juego indicando que el número de vidas es también de 5.
 - o Llama al método MuestraVidasRestantes de la nueva instancia y luego al de la instancia anterior.

